AlANA: Accelerating Additive Number Theoretic Transformation on GPU

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—The rapid transition toward Post-Quantum Cryptography (PQC) has catalyzed significant research into accelerating the Number Theoretic Transform (NTT), a cornerstone for efficient operations in prime fields. However, emerging cryptographic primitives, specifically those utilizing Binius, binary tower fields, and FRI-based commitment schemes, rely heavily on the Additive Number Theoretic Transform (ANTT). Despite its growing importance, ANTT acceleration remains less explored than its prime-field counterparts.**

In this paper, we propose a high-performance graphics processing unit (GPU)-based acceleration of the ANTT. By leveraging GPU architectural features such as shared memory for low-latency data access, memory tiling to optimize global memory bandwidth, and thread coarsening to increase work-per-thread efficiency, we minimize the computational bottlenecks inherent in binary field arithmetic. Furthermore, we implement in-memory computation techniques to reduce register pressure and synchronization overhead. Our implementation achieves a performance speedup of up to 1.3× compared to baseline GPU implementations. By demonstrating that ANTT can be substantially accelerated on modern GPUs, this work provides a foundation for scaling binary-field-based proof systems and highlights new opportunities enabled by emerging GPU architectural features such as tensor memory accelerators (TMA) and distributed shared memory (DSMEM).

***Index Terms*—Additive Number Theoretic Transforms, Binius, Cryptography, Fast Reed-Solomon Interactive Oracle Proof Acceleration.**

I. Introduction

Advances in post-quantum cryptography (PQC) have intensified the demand for fast polynomial evaluation and interpolation over large algebraic domains. While the classical Number Theoretic Transform (NTT) has been extensively optimized for prime fields and now serves as a core primitive in lattice-based schemes, a growing class of modern proof systems, including Binius [1], binary Reed–Solomon IOPs [2], and FRI-based polynomial commitment schemes [3], operate instead over binary extension fields. These systems rely fundamentally on the *Additive Number Theoretic Transform* (ANTT), an additive analogue of the NTT defined over $\mathbb{F}_{2^r}$ and structured around evaluations on nested additive subspaces. As these protocols scale to larger domains and higher rates, the ANTT becomes a dominant computational bottleneck.

Despite its increasing relevance, the ANTT has received far less attention than its multiplicative counterpart. Existing GPU implementations, such as the reference kernel in the `binius-gpu` [4] repository, prioritize correctness but do not exploit architectural features of modern accelerators. In contrast to the multiplicative NTT, whose twiddle factors are simple modular exponentiations, the ANTT requires evaluating linearized polynomials derived from subspace polynomials. These evaluations involve XOR-heavy arithmetic and data-dependent access patterns that do not map cleanly onto tensor cores or fused-multiply-add pipelines. As a result, techniques that have driven high-performance NTT implementations—including radix-k decomposition [5], multi-GPU scaling [6], and kernel fusion [7]—cannot be directly applied to the ANTT.

Nevertheless, the ANTT exhibits structural properties that make it a strong candidate for GPU acceleration. First, the butterfly pattern is highly regular: each stage consists of independent pairwise updates that can be parallelized across thousands of threads. Second, the transform is bandwidth-intensive, and GPUs offer significantly higher memory throughput than CPUs. Third, the ANTT's arithmetic is dominated by XOR and bitwise operations, which map efficiently to GPU ALUs. Finally, the evaluation domain is naturally partitioned into cosets, enabling coalesced memory access and shared-memory tiling strategies that reduce global memory traffic.

In this work, we present **AlANA**, a high-performance GPU implementation of the Additive Number Theoretic Transform. The main contributions of this work are:

C1. A tailored implementation of ANTT by combining (1) cooperative grid-wide synchronization for large butterfly stages, (2) warp-local shared-memory execution for small stages, and (3) thread coarsening to increase arithmetic intensity.

C2. In-register field multiplication to eliminate lookup-table stalls in binary field arithmetic.

C3. We evaluate AlANA in terms of runtime with the reference GPU implementation and achieve up to 2.61x improvement on the Ampere architecture and 4.25x improvement on the newer Blackwell architecture.

C4. We open-source our codes at https://github.com/SPIRE-GMU/binius-spire/tree/antt-alanahttps://github.com/SPIRE-GMU/binius-spire/tree/antt-alana for artifact evaluation and reproducible research.

II. Priliminaries

This section provides a brief overview of the background knowledge required to understand the ANTT, including binary fields, cantor basis, subspace polynomial, novel polynomial basis, lagrange basis over additive subspaces, and the additive number theoretic transformation.

A. Binary Fields

Finite fields are fundamental to modern cryptography. A field $(\mathbb{F}, +, \cdot)$ contains an additive identity 0, a multiplicative

0000–0000/00$00.00 © 2021 IEEE

identity 1, and has no zero divisisors: if $ab = 0$, then $a = 0$ or $b = 0$. Many modern proof systems, including Binius [1], operate over the binary field $\mathbb{F}_2 = \{0, 1\}$, where

$$a + b = a \oplus b, \qquad a \cdot b = a \wedge b.$$

To construct larger fields, Binius uses a quadratic tower of extensions:

$$\tau_m = \tau_{m-1}[X_{m-1}] \,/\, (X_{m-1}^2 + X_{m-1}X_{m-2} + 1), \qquad \tau_0 = \mathbb{F}_2,$$

where each extension introduces a new element x_k satisfying

$$x_k^2 = x_k x_{k-1} + 1.$$

An element $x \in \tau_m$ can be written as $x = L_x + R_x x_k$, and multiplication follows the recursive rule

$$xy = L_x L_y + (L_x R_y + L_y R_x)x_k + (R_x R_y)x_k^2,$$

with the final term reduced using the tower relation. This representation enables efficient bit-level arithmetic, making binary tower fields attractive for succinct arguments and polynomial IOPs [1].

B. Cantor Basis

Let $L = \mathbb{F}_{2^r}$ be an r-dimensional vector space over $\mathbb{F}_2$. A *Cantor basis* [8] is a recursively constructed basis

$$\{\beta_0, \beta_1, \ldots, \beta_{r-1}\},$$

where each β_i satisfies a quadratic relation. The Cantor basis enables efficient evaluation of linearized polynomials and is central to additive FFTs [9], [10].

Define the nested subspaces

$$U_i = \mathrm{span}_{\mathbb{F}_2}(\beta_0, \ldots, \beta_{i-1}), \qquad |U_i| = 2^i.$$

C. Subspace Polynomials

For each subspace U_i, the associated subspace polynomial is

$$W_i(x) = \prod_{u \in U_i} (x + u).$$

Because the field has characteristic 2, W_i is a linearized polynomial of the form

$$W_i(x) = \sum_{j=0}^{i} a_j x^{2^j}.$$

A normalized version,

$$\hat{W}_i(x) = \frac{W_i(x)}{W_i(\beta_i)},$$

satisfies

$$\hat{W}_i(x) = \begin{cases} 0, & x \in U_i, \\ 1, & x \in U_i + \beta_i. \end{cases}$$

These polynomials form the backbone of additive FFTs and ANTT constructions [9], [10].

D. Novel Polynomial Basis

Let $\mathcal{R}$ be the space of polynomials of degree $< 2^\ell$. Instead of the monomial basis, we use the *novel basis* introduced by Lin et al. [11]:

$$X_i(x) = \prod_{k=0}^{\ell-1} \hat{W}_k(x)^{b_k}, \qquad i = \sum_{k=0}^{\ell-1} b_k 2^k.$$

This basis aligns with the additive subspace structure and enables divide-and-conquer evaluation. Any polynomial $D(x)$ can be written as

$$D(x) = \sum_{m=0}^{2^\ell - 1} d_m X_m(x).$$

The novel basis has been used extensively in efficient Reed–Solomon encoding and decoding [2], [11] and is foundational to modern additive FFT constructions [9], [10].

E. Lagrange Basis Over Additive Subspaces

The evaluation domain of the ANTT is the additive subgroup U_ℓ. The Lagrange basis associated with U_ℓ is

$$\{\delta_u(x)\}_{u \in U_\ell},$$

where $\delta_u(x)$ is 1 at $x = u$ and 0 at all other points in U_ℓ. The ANTT converts coefficients in the novel basis into evaluations in this Lagrange basis.

F. Additive Number Theoretic Transform (ANTT)

The ANTT is the additive analogue of the classical NTT, replacing multiplicative roots of unity with additive subspaces and linearized polynomials [9], [10]. Define the recursive operator $\Delta_{m,i}(x)$:

$$\Delta_{m,\ell}(x) = d_m,$$

$$\Delta_{m,i}(x) = \Delta_{m,i+1}(x) + \hat{W}_i(x)\, \Delta_{m+2^i,i+1}(x).$$

At each stage, the butterfly update is

$$x_0 = y_0 + w y_1, \qquad x_1 = x_0 + y_1,$$

where $w = \hat{W}_i(\alpha)$ for the appropriate coset representative α.

By induction,

$$\Delta_{0,0}(x) = D(x),$$

yielding all evaluations $D(u)$ for $u \in U_\ell$. Figure 1 illustrates the data flow for a 4-point ANTT.

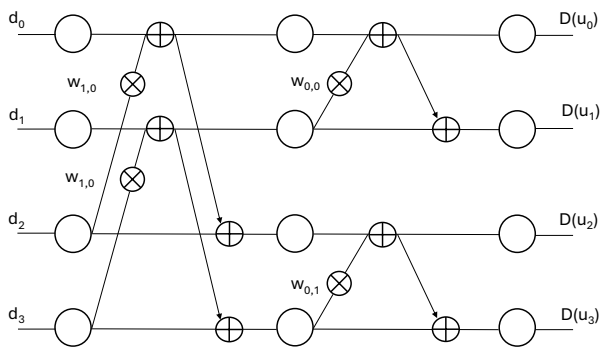

Fig. 1. Butterfly diagram for a 4-point Additive Number Theoretic Transform (ANTT). Each box represents a butterfly combining two partial evaluations using the appropriate twiddle factor $W_{i,j}$.

III. Related Work

The multiplicative Number Theoretic Transform (NTT) has been extensively optimized on GPUs, resulting in a mature ecosystem of high-performance implementations. Early work such as Øzerk et al. [12] demonstrated that shared-memory tiling and coalesced memory layouts significantly reduce global memory traffic on consumer GPUs. More recent efforts push performance further on modern architectures: Ozcan et al. [5] achieve sub-10 μs execution for 2^{14}-point NTTs on the RTX 4090 using radix-2 Cooley–Tukey and 4-step algorithms, while Ji et al. [6] show that multi-GPU scaling can accelerate zero-knowledge proof systems by distributing NTT workloads across devices. Other works, such as NTTFusion [7], explore kernel fusion and memory-layout manipulation to reduce kernel-launch overhead and improve data locality.

Collectively, these results show that the multiplicative NTT is a well-studied and highly optimized primitive on GPUs. Techniques such as radix-k decomposition, 4-step algorithms, shared-memory staging, kernel fusion, and tensor-core acceleration have all been successfully applied to reduce latency and improve throughput.

In contrast, the Additive Number Theoretic Transform (ANTT) has received almost no attention in the GPU literature, despite its growing importance in modern cryptographic systems. ANTT is a key component in binary Reed–Solomon codes [2], Binius-style polynomial commitments [1], and additive FFT constructions [9], [10]. These protocols operate over binary extension fields and rely on linearized polynomials and additive subspaces, structures fundamentally different from the multiplicative roots of unity used in classical NTTs.

The only publicly available GPU implementation of ANTT is the reference kernel in the `binius-gpu` repository, which prioritizes correctness over performance and does not exploit shared-memory tiling, warp-level primitives, kernel fusion, or memory-layout optimizations. As a result, substantial performance headroom remains unaddressed.

Although ANTT and NTT share a butterfly structure, several core differences prevent direct reuse of high-performance NTT techniques. First, tensor-core acceleration is effective for multiplicative NTTs due to their reliance on modular multiplication, whereas it does not apply to ANTT, whose arithmetic is dominated by XOR operations and evaluations of linearized polynomials. These operations do not map naturally to matrix-multiply-accumulate pipelines. Second, radix-k decompositions (e.g., radix-4, radix-8, radix-16), widely used to accelerate NTTs, are incompatible with ANTT. In multiplicative NTTs, butterflies within a radix block are independent, enabling parallel updates. In ANTT, however, each butterfly requires a sequential dependency:

$$u' = u + wv, \qquad v' = u' + v,$$

meaning v' cannot be computed until u' is known. This dependency forces ANTT to remain strictly radix-2. Finally, polynomial reductions in binary tower fields introduce additional data dependencies and irregular access patterns that do not appear in prime-field NTTs.

Given the cryptographic relevance of ANTT in Binius [1], generalized Reed–Solomon codes [2], and additive FFTs [9], [10], and the absence of optimized GPU implementations, ANTT represents a timely and impactful target for acceleration. Our work addresses this gap by designing the first high-performance GPU ANTT, leveraging shared-memory tiling, memory coalescing, thread coarsening, and multi-kernel staging tailored specifically to the additive transform's algebraic structure.

IV. Methodology

A. Baseline CPU Implementation

We first implemented a CPU version of the Additive Number Theoretic Transform (ANTT) to understand the computational structure of the transform. This reference implementation made explicit the stage-by-stage dependency pattern, the butterfly update rule

$$u' = u + wv, \qquad v' = u' + v,$$

and the role of the twiddle factors derived from subspace polynomials. The CPU version also revealed that each stage must complete globally before the next begins, as the butterfly introduces a strict data dependency.

B. Identifying Bottlenecks

Two primary bottlenecks emerged from the CPU analysis and initial GPU prototypes:

a) Stage-Level Synchronization.: ANTT requires strict stage ordering. A naive GPU implementation would require a kernel launch per stage or device-wide synchronization, both of which incur high overhead. Cooperative grid launches enable efficient grid-wide synchronization and allow thread coarsening, improving arithmetic intensity and reducing scheduling overhead.

b) LUT-Based Field Multiplication.: In fields such as $\mathbb{F}_{2^4}$, multiplication is often implemented using a 256-entry lookup table. When thousands of GPU threads simultaneously access this table, cache thrashing and memory serialization occur. We replaced the LUT with in-register polynomial multiplication modulo the irreducible polynomial, eliminating memory stalls.

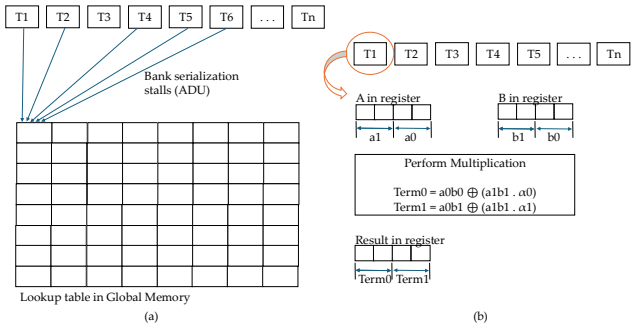

Fig. 2. Comparison of finite field multiplication strategy. (a) The baseline uses lookup table to get the multiplication result which results in bank serialization stalls increasing the address divergence unit utilization while ALU remains stalled. (b) AlANA's approach loads the values in register and improves the ALU utilization.

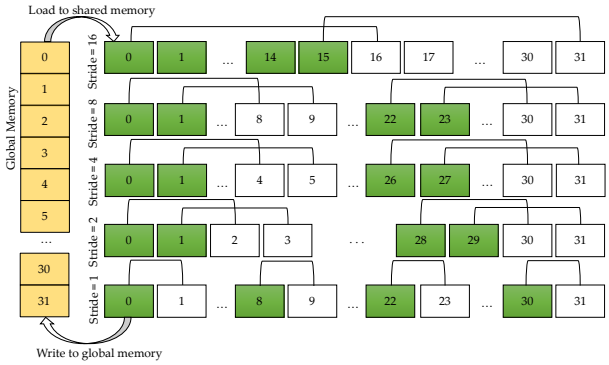

Fig. 3. A single warp loads a data tile from global to shared memory, executes five successive butterfly stages (Stride 16 to 1) locally, and writes the transformed results back.

C. GPU Optimization Strategy

1) Coalesced Global Memory Access: We reorganized memory access so that each warp loads 32 consecutive elements, ensuring fully coalesced global memory transactions and maximizing bandwidth utilization.

2) Memory Tiling and Double Buffering: For the upper stages of the ANTT, where butterfly partners are far apart, we use the `coop_antt` kernel. Algorithm 1 describes how it employs shared-memory tiling, double buffering, asynchronous global memory loads, and grid-wide synchronization. This kernel overlaps memory transfers with computation and hides global memory latency.

Algorithm 1 `AlANA_COOP`: Cooperative ANTT Kernel (Upper Stages)

Require: data matrix D, constants C, stage range $[\text{end}, \text{start}]$
0: Partition the shared memory to support async copy of next tile
0: Load tile of u and v values into shared memory (async)
0: **for** stage $=$ start down to end **do**
0: Wait for async copy to complete
0: Issue load next tile of u and v values into shared memory (async)
0: **for** each thread t in the block **do**
0: Compute global index i and partner $j = i \oplus (1 \ll \text{stage})$
0: Twiddle $w = \textsc{ComputeTwiddle}(C, \text{stage}, \text{block}(i))$
0: Perform butterfly: $u \leftarrow u + wv$, $v \leftarrow u + v$
0: **end for**
0: Write updated tile back to global memory (async)
0: **grid_sync()**
0: **end for**=0

3) Shared-Memory Execution for Small Stages (AlANA_inwarp): For the lower stages (stage 4 down to stage 0), butterfly partners lie within a small contiguous region. These stages fit entirely within a warp's shared memory tile. This eliminates global memory traffic and synchronization for the final stages. Algorithm 2 shows the steps to compute the small stages in warp. Although 50% of the threads wait for the synchronization, it does not affect the warp scheduling by the SM. The process of processing small stages in shared memory is illustrated in Figure 3, which shows how the in-memory execution strategy eliminates global synchronization overhead for the final ANTT stages.

Algorithm 2 `AlANA_inwarp`: Warp-Local ANTT Kernel (Lower Stages)

Require: shared tile S, constants C, stage range $[0, s_{\max}]$
0: Load tile from global memory into shared memory
0: **for** stage $= s_{\max}$ down to 0 **do**
0: **for** each thread t in the warp **do**
0: $i \leftarrow t, \quad j \leftarrow t \oplus (1 \ll \text{stage})$
0: **if** $j > i$ **then**
0: Twiddle $w = \textsc{ComputeTwiddle}(C, \text{stage}, \text{block}(i))$
0: $u \leftarrow S[i], \quad v \leftarrow S[j]$
0: $u \leftarrow u + wv, \quad v \leftarrow u + v$
0: $S[i] \leftarrow u, \quad S[j] \leftarrow v$
0: **end if**
0: **end for**
0: **syncthreads()**
0: **end for**
0: Write shared tile back to global memory =0

The full ANTT is executed by splitting the stages into two regions, the steps are illustrated in Algorithm 3. This two-kernel decomposition balances throughput and latency, achieving high performance across all stages of the ANTT.

Algorithm 3 Kernel Launch Flow for Full ANTT

Require: $\log h$, $\log r$, data D
0: $s_{\text{inwarp}} \leftarrow \min(4, \log h - 1)$
0: **// Upper stages: cooperative kernel**
0: **if** $\log h - 1 > s_{\text{inwarp}}$ **then**
0: Launch `coop_antt` with stages $[\log h - 1, s_{\text{inwarp}} + 1]$
0: **end if**
0: **// Lower stages: warp-local kernel**
0: Launch `antt_inwarp` with stages $[s_{\text{inwarp}}, 0]$ =0

TABLE I
RUNTIME COMPARISON ACROSS RTX 3090 TI (AMPERE), JETSON THOR (BLACKWELL), AND NVIDIA A100 (AMPERE) FOR BLOWOUT FACTOR 0.

	RTX 3090 Ti (Ampere)			Jetson Thor (Blackwell)			NVIDIA A100 (Ampere)		
log h	**Ref** (ms)	**AlANA** (ms)	**Speedup**	**Ref** (ms)	**AlANA** (ms)	**Speedup**	**Ref** (ms)	**AlANA** (ms)	**Speedup**
21	6.9062	5.5589	1.24	29.1765	23.9161	1.22	6.702	5.935	1.13
22	13.0220	10.4913	1.24	60.4115	48.0637	1.26	13.276	11.57	1.15
23	26.7741	21.9469	1.22	125.7779	99.2369	1.27	27.509	23.525	1.17
24	54.0044	46.1545	1.17	261.5424	205.1331	1.27	57.057	45.869	1.24
25	112.0502	89.5819	1.25	555.0758	423.8463	1.31	116.404	93.239	1.25
26	223.5556	168.9512	1.32	1163.6286	874.9792	1.33	228.893	173.113	1.32
27	468.4398	341.7753	1.37	2432.1157	1807.3216	1.35	477.896	356.058	1.34
28	976.1361	708.3755	1.38	5076.6716	3717.1000	1.37	1006.381	743.369	1.35
29	1975.0510	1433.5075	1.38	10423.1729	7612.3578	1.37	2055.063	1514.061	1.36
30	4051.2041	2885.9442	1.40	21396.8418	15645.4250	1.37	4206.739	3104.717	1.36

TABLE II
RUNTIME COMPARISON ACROSS RTX 3090 TI (AMPERE), JETSON THOR (BLACKWELL), AND NVIDIA A100 (AMPERE) FOR BLOWOUT FACTOR 2.

	RTX 3090 Ti (Ampere)			Jetson Thor (Blackwell)			NVIDIA A100 (Ampere		
log h	**Ref** (ms)	**AlANA** (ms)	**Speedup**	**Ref** (ms)	**AlANA** (ms)	**Speedup**	**Ref** (ms)	**AlANA** (ms)	**Speedup**
19	5.9130	5.0515	1.17	27.2862	21.9160	1.25	6.165	5.595	1.10
20	12.7046	9.9784	1.27	56.7092	44.9705	1.26	11.568	10.746	1.08
21	26.4990	20.6389	1.28	118.5308	93.2382	1.27	23.713	20.524	1.16
22	52.4460	39.4211	1.33	249.3962	193.0560	1.29	49.083	39.054	1.26
23	107.5193	80.8316	1.33	528.5537	399.6048	1.32	103.863	79.53	1.31
24	219.5171	153.5723	1.43	1112.2304	825.1069	1.35	218.294	163.21	1.34
25	450.1499	316.5041	1.42	2302.8816	1710.2819	1.35	451.344	336.316	1.34
26	905.2308	664.3625	1.36	4750.2374	3524.5183	1.35	939.15	703.772	1.34
27	1853.6914	1359.6623	1.36	9770.6010	7228.1823	1.35	1921.508	1433.922	1.34

V. EXPERIMENTAL EVALUATION

We evaluate our GPU-accelerated ANTT implementation on two architectures: (1) Ampere (NVIDIA RTX 3090 Ti and NVIDIA A100) , and (2) Blackwell (NVIDIA Jetson Thor). The former represents a high-end desktop GPU, while the latter represents NVIDIA's newest embedded Blackwell-class accelerator. For each configuration, we executed five independent runs and report the average runtime. We evaluate input sizes $n = 10$ to $n = 30$ and blowup factors $\{0, 2\}$.

Table I and Table II report the runtime of the reference baseline and our optimized implementation on the RTX 3090 Ti, Jetson Thor, and NVIDIA A100 for different log levels of input for blowout factors 0 and 2, respectively. For blowout factor 0, the speedup gain improves with the size of the input on every device. For blowout factor 2, the improvement on the RTX 3090 Ti peaks for input size 2^{24}, gaining a speedup of $1.43\times$. For the NVIDIA A100, the result was different from that of the RTX 3090 Ti and Jetson Thor for the lower input sizes.

VI. CONCLUSION

This work presents **AlANA**, a high-performance GPU implementation of the ANTT. While the multiplicative NTT has benefited from more than a decade of GPU optimization, the ANTT has remained largely unexplored despite its growing importance in binary-field proof systems, Binius-style polynomial commitments, and modern Reed–Solomon constructions. By analyzing the algebraic structure of the ANTT and the execution characteristics of contemporary NVIDIA architectures, we design a two-kernel pipeline that exploits both large-stage and small-stage parallelism. As binary-field cryptography continues to gain traction, fast ANTT implementations will become increasingly critical. Our work establishes a foundation for future research in GPU-accelerated additive transforms, including opportunities to explore deeper kernel fusion, DSMEM-enabled multi-stage pipelines, and integration with full Binius and FRI provers.

REFERENCES

[1] B. E. Diamond and J. Posen, "Succinct arguments over towers of binary fields," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 93–122, Springer, 2025.

[2] X. Zheng and X. Ma, "On generalized reed-solomon codes," *IEEE Transactions on Communications*, vol. 73, no. 11, pp. 9976–9986, 2025.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed-Solomon Interactive Oracle Proofs of Proximity," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)* (I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella, eds.), vol. 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, (Dagstuhl, Germany), pp. 14:1–14:17, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.

[4] IrreducibleOSS, "binius-gpu." https://github.com/IrreducibleOSS/binius-gpu, 2026. GitHub repository. Accessed: 2026-05-13.

[5] A. Ozcan, A. Javeed, and E. Savas, "High-performance number theoretic transform on gpu through radix2-ct and 4-step algorithms," *IEEE Access*, vol. 13, pp. 87862–87883, 2025.

[6] Z. Ji, J. Zhao, P. Gao, X. Yin, and L. Ju, "Accelerating number theoretic transform with multi-gpu systems for efficient zero knowledge proof," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, (New York, NY, USA), p. 1–14, Association for Computing Machinery, 2025.

[7] Z. Wang, P. Li, R. Hou, and D. Meng, "Nttfusion: Efficient number theoretic transform acceleration on gpus," in *2023 IEEE 41st International Conference on Computer Design (ICCD)*, pp. 357–365, 2023.

[8] D. G. Cantor, "On arithmetical algorithms over finite fields," *Journal of Combinatorial Theory, Series A*, vol. 50, no. 2, pp. 285–300, 1989.

[9] S. Gao and T. Mateer, "Additive fast fourier transforms over finite fields," *IEEE Transactions on Information Theory*, vol. 56, no. 12, pp. 6265–6272, 2010.

[10] W.-D. Li, M.-S. Chen, P.-C. Kuo, C.-M. Cheng, and B.-Y. Yang, "Frobenius additive fast fourier transform," in *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*, ISSAC '18, (New York, NY, USA), p. 263–270, Association for Computing Machinery, 2018.
[11] S.-J. Lin, W.-H. Chung, and Y. S. Han, "Novel polynomial basis and its application to reed-solomon erasure codes," in *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pp. 316–325, 2014.
[12] Ö. Özerk, C. Elgezen, A. C. Mert, E. Öztürk, and E. Savaş, "Efficient number theoretic transform implementation on gpu for homomorphic encryption," *The Journal of Supercomputing*, vol. 78, pp. 2840–2872, 2 2022.